

Loesungen — Blatt 1

Aufgaben: [\[\[numerik/exercises/01/exercise-01|Uebung 1\]\]](#) **PDF:** [\[\[numerik/exercises/01/solution-01.pdf|solution-01.pdf\]\]](#) **Quellcode:** [numerik/repos/numerik/blatt01/](#)

Inhaltsverzeichnis

- [\[\[#Aufgabe 1 — pq-Formel vs. Vieta|Aufgabe 1 — pq-Formel vs. Vieta\]\]](#)
 - [\[\[#Aufgabe 2 — Reihenentwicklung \$\ln\(1+x\)\$ |Aufgabe 2 — Reihenentwicklung \$\ln\(1+x\)\$ \]\]](#)
 - [\[\[#Aufgabe 3 — Numerische Integration|Aufgabe 3 — Numerische Integration\]\]](#)
 - [\[\[#Aufgabe 4 — Vektorisierte Trapezregel|Aufgabe 4 — Vektorisierte Trapezregel\]\]](#)
 - [\[\[#Aufgabe 5 — Wurzelberechnung|Aufgabe 5 — Wurzelberechnung\]\]](#)
-

Aufgabe 1 — pq-Formel vs. Vieta

```
import numpy as np

def pq_formel(p, q):
    return -p + np.sqrt(p*p + q)

def vieta(p, q):
    x1 = -p - np.sqrt(p*p + q)
    return -q / x1

q = 1
for exp in [2, 4, 6, 7, 8]:
    p = 10**exp
    print(f"p={p}: pq={pq_formel(p,q):.10f}  vieta={vieta(p,q):.10f}")
```

Ergebnis: Fuer grosse p verliert die pq-Formel Genauigkeit (Ausloeschung bei Subtraktion fast gleich grosser Zahlen). Das Vieta-Verfahren ist fuer grosse p zuverlaessiger.

Aufgabe 2 — Reihenentwicklung $\ln(1+x)$

```
import numpy as np

def ln_reihe(x, n, dtype):
    s, xk = dtype(0), dtype(x)
```

```

x = dtype(x)
for k in range(1, n + 1):
    s += dtype((-1)**(k+1)) * xk / dtype(k)
    xk *= x
return s

x, n = 0.9, 50
print(f"exact: {np.log(1+x):.10f}")
print(f"float16: {float(ln_reihe(x, n, np.float16)):.10f}")
print(f"float64: {ln_reihe(x, n, np.float64):.10f}")

```

Ergebnis: float64 liefert gute Genauigkeit, float16 zeigt bereits ab der 2. Dezimalstelle Abweichungen.

Aufgabe 3 — Numerische Integration

```

import numpy as np

def rechtecksumme(f, a, b, n):
    h = (b - a) / n
    return h * sum(f(a + i*h) for i in range(n))

def trapezregel(f, a, b, n):
    h = (b - a) / n
    return h/2 * (f(a) + 2*sum(f(a + i*h) for i in range(1, n)) + f(b))

# Test: int(1/x^2, 0.1, 10) = 9.9 und int(ln(x), 1, 2) = 2ln2 - 1
for n in [10, 100, 1000]:
    print(f"n={n}: RS={rechtecksumme(lambda x: 1/x**2, 0.1, 10, n):.10f}"
          f" T={trapezregel(lambda x: 1/x**2, 0.1, 10, n):.10f}")

```

Ergebnis: Trapezregel konvergiert schneller als Rechtecksumme.

Aufgabe 4 — Vektorisierte Trapezregel

```

import numpy as np

def trapez_vekt(f, a, b, n):
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    return h/2 * (y[0] + 2*np.sum(y[1:-1]) + y[-1])

def rechtsumme_vekt(f, a, b, n):

```

```

h = (b - a) / n
x = np.linspace(a, b, n + 1)
return h * np.sum(f(x)[1:])

```

Ergebnis: Identische Resultate wie Aufgabe 3, aber deutlich schneller durch NumPy-Vektoroperationen.

Aufgabe 5 — Wurzelberechnung

(a) Taylor-Entwicklung

```

import numpy as np

def sqrt_taylor(x, x0, steps=50):
    a = y = np.sqrt(x0)
    for k in range(1, steps + 1):
        a = (3/(2*k) - 1) * (x/x0 - 1) * a
        y += a
    return y

# sqrt(2): x0=1 → langsame Konvergenz, x0=4 → schnellere Konvergenz

```

(b) Heron-Verfahren

```

def sqrt_heron(x, y0, steps=50):
    y = y0
    for _ in range(steps):
        y = 0.5 * (y + x/y)
    return y

# sqrt(2): Konvergenz < 0.005 nach 2-3 Schritten (vs. ~5-10 bei Taylor)

```

Ergebnis: Heron konvergiert **quadratisch** (Fehler wird pro Schritt quadriert), Taylor nur linear. Heron erreicht Abweichung < 0.005 in 2-3 Schritten, Taylor benötigt deutlich mehr.